

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers using nano file name as practical 6

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <string.h>

#define FIFO_REQ "fifo_request"
#define FIFO_RESP "fifo_response"
#define MSG_LEN 64

int main()
{
    unlink(FIFO_REQ);
    unlink(FIFO_RESP);

    mkfifo(FIFO_REQ, 0666);
    mkfifo(FIFO_RESP, 0666);

    pid_t client1 = fork();

    if (client1 == 0)
    {
        int fd_req = open(FIFO_REQ, O_WRONLY);
        write(fd_req, "Client 1: Task A", 17);
        close(fd_req);

        char reply[MSG_LEN];
        int fd_resp = open(FIFO_RESP, O_RDONLY);
        read(fd_resp, reply, sizeof(reply));
        printf("[Client 1] Received Reply: %s\n", reply);
        close(fd_resp);

        exit(0);
    }

    pid_t client2 = fork();
```

```

if (client2 == 0)
{
    usleep(100000);
    int fd_req = open(FIFO_REQ, O_WRONLY);
    write(fd_req, "Client 2: Task B", 17);
    close(fd_req);

    char reply[MSG_LEN];
    int fd_resp = open(FIFO_RESP, O_RDONLY);
    read(fd_resp, reply, sizeof(reply));
    printf("[Client 2] Received Reply: %s\n", reply);
    close(fd_resp);

    exit(0);
}

for (int i = 0; i < 2; i++)
{
    char buffer[MSG_LEN];
    int fd_req = open(FIFO_REQ, O_RDONLY);
    read(fd_req, buffer, sizeof(buffer));
    close(fd_req);

    printf("[Server] Processed Request: %s\n", buffer);

    int fd_resp = open(FIFO_RESP, O_WRONLY);
    char ack[MSG_LEN];
    snprintf(ack, sizeof(ack), "ACK for (%s)", buffer);
    write(fd_resp, ack, strlen(ack) + 1);
    close(fd_resp);
}

waitpid(client1, NULL, 0);
waitpid(client2, NULL, 0);

unlink(FIFO_REQ);
unlink(FIFO_RESP);

return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

```

```

#include <unistd.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\n[Handled] Caught SIGINT (Signal 2)\n");
    else if (sig == SIGUSR1)
        printf("[Handled] Caught SIGUSR1 (Signal 30/10)\n");
    else if (sig == SIGTERM)
    {
        printf("[Handled] Caught SIGTERM (Signal 15). Exiting gracefully!\n");
        exit(0);
    }
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGUSR1, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("Starting self-testing signal program...\n");
    sleep(1);

    printf("Sending SIGINT to myself...\n");
    raise(SIGINT);
    sleep(1);

    printf("Sending SIGUSR1 to myself...\n");
    raise(SIGUSR1);
    sleep(1);

    printf("Sending SIGTERM to myself...\n");
    raise(SIGTERM);

    return 0;
}

```